

2025 Jan Oppe 1 - Set 2

Section 1 - problem 2

Check String Rotation

PROBLEM STATEMENT

Write a function that checks if one string is a rotation of another.

Hint: all the rotations of a particular string will be present in the string formed by concatenating it with itself.



EXAMPLE & EXPLANATION

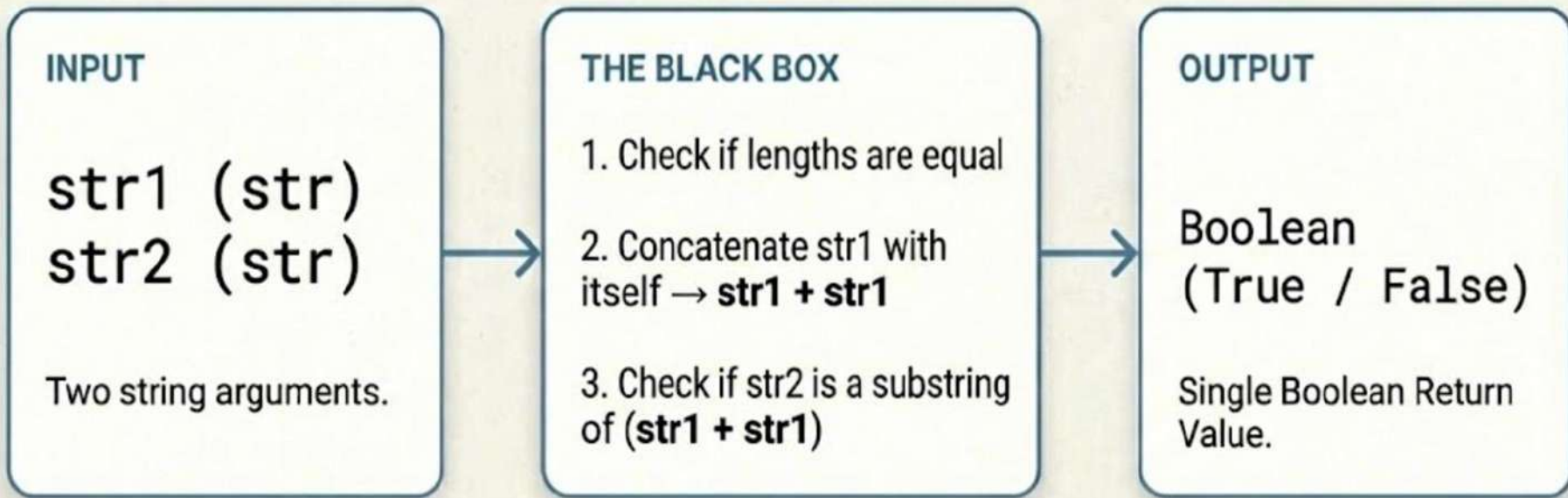
Example:

```
>>>is_rotation('abcde', 'cdeab') True  
>>>is_rotation('hello', 'ehllo') False  
>>>is_rotation('hello', 'helol') False
```

Explanation:

Since **cdeab** is a rotation of **abcde**,
So, **is_rotation('abcde', 'cdeab')** should
return True

THE PROBLEM SPECIFICATION

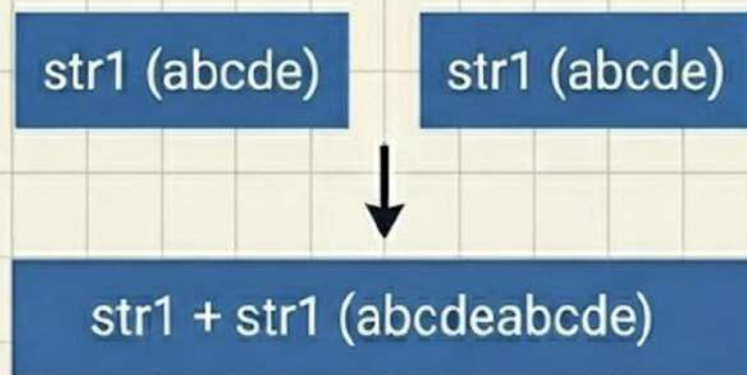


Constraint: If lengths are different → immediately return False

Visualizing String Rotation

Example Data: str1 = 'abcde', str2 = 'cdeab'

Concatenate 'str1' with itself



Resulting String: 'abcdeabcde'

Check if 'str2' is a substring

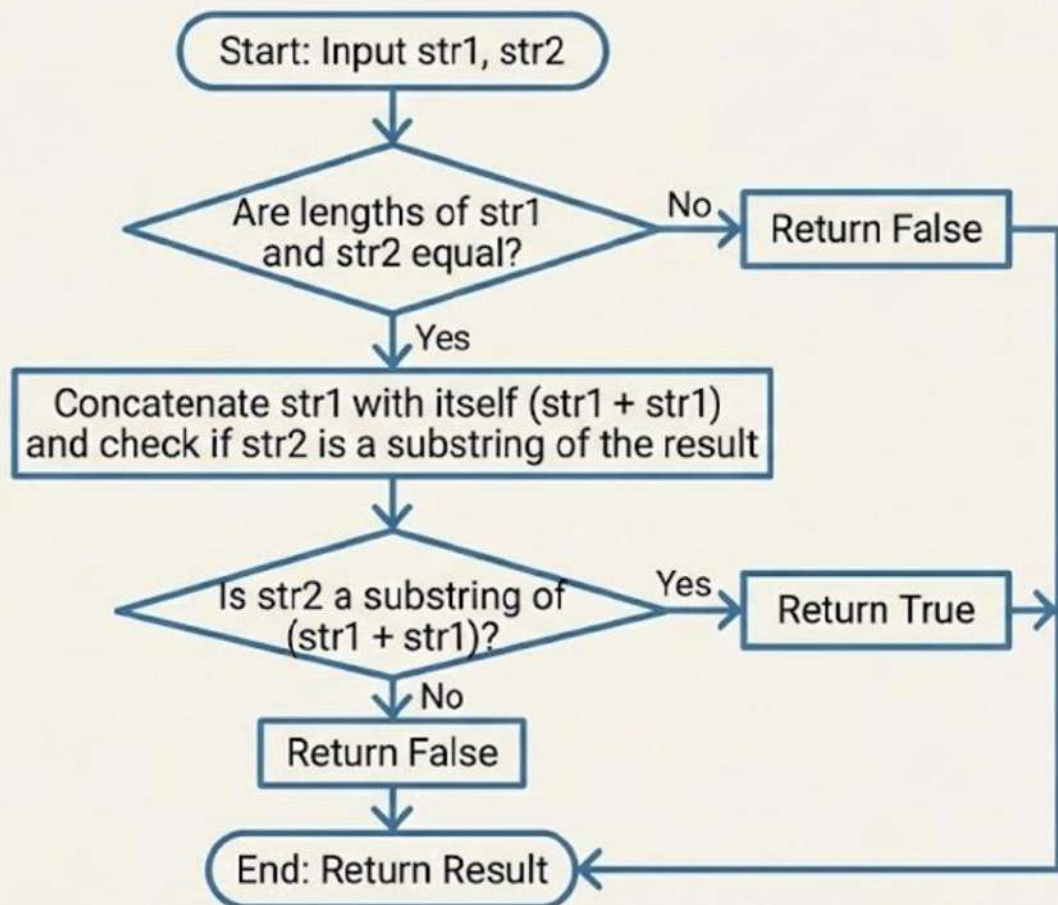


'cdeab' is a substring of 'abcdeabcde'

Substring found

Result: Overall: True

The Logic Flowchart



The Explanatory Approach

Step-by-Step implementation for clarity.

```
def is_rotation(str1, str2):  
    # Step 1: Check if lengths are equal  
    len1 = len(str1)  
    len2 = len(str2)  
    if len1 != len2:  
        return False  
  
    # Step 2: Concatenate str1 with itself  
    concatenated_str = str1 + str1  
  
    # Step 3: Check if str2 is a substring  
    is_substring = str2 in concatenated_str  
    return is_substring
```

Rotations must have the same length.

Creates a string containing all possible rotations.

Returns True if str2 is found, otherwise False.

The Pythonic Optimization

Leveraging built-ins for concise logic.

Type Hinting
for clarity.

Substring check in
concatenated string.

```
def is_rotation(str1: str, str2: str) -> bool:  
    return len(str1) == len(str2) and str2 in (str1 + str1)
```

Concise one-liner combines
length and substring checks.

Result: Identical output, cleaner syntax

Concept Spotlight: String Concatenation Trick

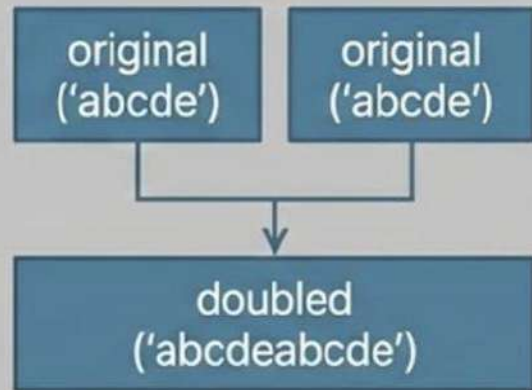
Doubling a string to check for rotated substrings.

Appending a string to itself (`str + str`) contains all possible rotations as substrings. This simplifies rotation checks to a single substring lookup.

Code View

```
original = "abcde"  
doubled = original + original
```

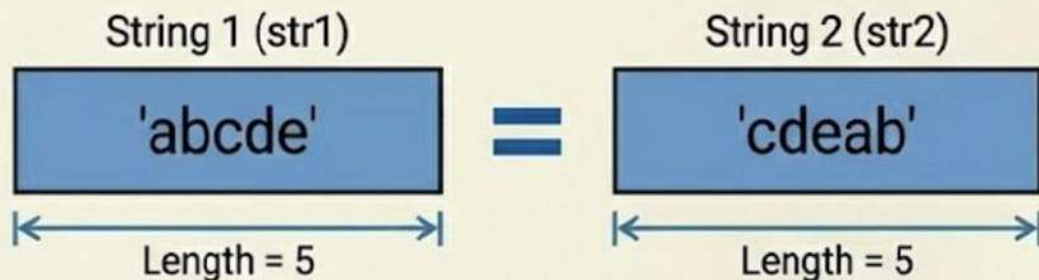
Runtime Object



Application: 'cdeab' in ('abcde' + 'abcde') → True

Concept Spotlight: Importance of Length Check

Ensuring strings are candidates for rotation by verifying they have the same length.



Status: **MATCH** ✓ (Proceed to rotation check)

Scenario A: Unequal Lengths

str1 = 'abc', str2 = 'ab'
len(str1) (3) != len(str2) (2)
Length check fails.
Result: False (Immediate)

Scenario B: Equal Lengths, Not Rotation

str1 = 'abc', str2 = 'acb'
len(str1) (3) == len(str2) (3)
Length check passes. Proceed to rotation check.
Rotation check fails.
Result: False

Scenario C: Equal Lengths, Is Rotation

str1 = 'abc', str2 = 'bca'
len(str1) (3) == len(str2) (3)
Length check passes. Proceed to rotation check.
Rotation check passes.
Result: True

Robustness Rule: Always perform a **length check** first to optimize performance and avoid unnecessary complex rotation calculations.

Solution Comparison: Check String Rotation

Step-by-Step Approach

- Utilizes intermediate variables for clarity.
- Easier to debug and understand line-by-line.

```
def is_rotation(str1, str2):  
    # Step 1: Check if lengths are equal  
    lengths_equal = len(str1) == len(str2)  
  
    # Step 2: Concatenate str1 with itself  
    doubled_str1 = str1 + str1  
  
    # Step 3: Check if str2 is a substring  
    is_substring = str2 in doubled_str1  
  
    # Step 4: Return the result of both conditions  
    return lengths_equal and is_substring
```

Concise One-Liner

- More 'Pythonic' and efficient.
- Reduces visual noise with less code.

```
def is_rotation(str1: str, str2: str) -> bool:  
    # Concise one-liner combining length  
    # and substring checks  
    return len(str1) == len(str2) and str2  
           in (str1 + str1)
```

The logical ``and`` operator returns ``True`` only if both the length check and the substring check are ``True``.

Key Takeaways

01

Problem: String Rotation Check

Determine if a second string (str2) is a cyclic rotation of the first string (str1).

02

Core Logic: Concatenation Trick

If lengths are equal, all rotations of str1 are present as substrings in the concatenated string (str1 + str1).

03

Return Value: Boolean

The function should return 'True' if str2 is a rotation of str1, and 'False' otherwise.

Final Syntax Cheat Sheet

```
return len(str1) == len(str2) and str2 in (str1 + str1)
```